Une : a Declarative DSL for the Mingle Platform

MISCELLANEOUS

- Language is case **in**sensitive
- Command separator: LF + LF (blank line)
- Comments: # (ignores to the end of the line)
- \ continues in next line
- Commands can be in any order(1)
- Compiler always use UTF-8 for output
- Escaped (inside strings): \", \\, \n, \r, \t

(1) The same for the clauses inside a command



DATA TYPES

BASIC

- BOOLEAN: { TRUE | FALSE }
  (Alias: CLOSED, ON, YES | OPEN, OFF, NO)
- NUMBER : Decimal (signed 32-bits: 1234.56 or 1_234.56)
- STRING : Everything in between double quotes (“ ”)

EXTENDED
date(), time(), list(), pair()

NUMBERS SUFFIXES

Time : { u | t | s | m | h | d } (converts to milliseconds)

Temperature: { c | f | k } (converts to Celsius)

CONVENTIONAL OPERATORS

- Arithmetic: +, -, *, /, %, ^
  (Sum, Diff, Times, Divide,Mod, Power)
- Relational: ==, >, <, >=, <=, !=, <>
  (Alias: IS or EQUALS or ARE, BELOW, ABOVE, LEAST, MOST, UNEQUAL, IS_NOT)
- Conditional: &&, ||, |&, !
  (Alias: AND, OR, XOR, NOT)
- Bitwise: &, |, ><, ~ >> <<
  (Alias: BAND, BOR, BXOR, BNOT) (shifts)
- Assign: = (Alias: SET)
- Send (functions invocation): :
- Name-space separator (dot): .
- Quantified: ANY, ALL, NONE

MACROS FOLDERS

- {*home*} MSP home
- {*home.inc*} includes
- {*home.lib*} libraries
- {*home.log*} logs
- {*home.tmp*} temporal
- {*home.etc*} others

EDGE OPERATORS

- ?> RISES (cur > t AND prev <= t)
- ?< DROPS (cur < t AND prev >= t)
- ?= BECOMES (cur == v AND prev != v)
- ?!= LEAVES (cur != v AND prev == v)
- ?<> CROSSES (crosses either way)

cur → current value prev → previous value

ENUNCIATIONS

<name[1]> : Letter, Digit[2] or ‘_‘ (max 48 chars)

<type> : Either a ‘basic’ or a ‘extended’ data type

<expression>: { <name> | <type> } <operator> <expression>

<literal> : <name> | <type> | <expression>

(1) In *Une*, ‘name’ is the same as ‘identifier’ or ‘label’ in other languages
(2) Names can not start with a digit

PREPROCESSOR

INCLUDE {“<URI [*|**](1)>” | “*”} [USE <name> [, …] AS <literal>[, … ;]]

USE <literal> AS <literal> [; …](2)

USE <name> AS {<literal> | <string>} [; …](2)

USE <name> <param1> [literal] <param2> …
AS <single-command_expansion>(3)

(1) * Includes all files in folder, ** includes also subfolders
(2) Plain symbol replacement
(3) Parameterized template

SCRIPTS (for advanced users)

SCRIPT [<name>]
LANGUAGE { java | js | python | ...}
FROM { “<URI>” [; ...] | {<code>} }
[CALL “<entry_point>”]
[[PRE] ONSTART] [[PRE] ONSTOP]

DRIVERS (for advanced users)

DRIVER <name>
SCRIPT <script-name>
[CONFIG <prop> AS {ANY | <data_type>} [REQUIRED] [; ...]]

LIBRARY (for medium and advanced users)

LIBRARY <name>
LANGUAGE(1) { jvm | js | python | ...}
[FROM(2) “<URI>” [; ...]]
[CONFIG name = {<literal> | <expression(3)>} [; …]]

Currently supported languages in Mingle:
JVM (also Java), Python, JavaScript, Ruby, R, Native (C/C++ and Rust and .so, .dll, .dylib)

(1) All languages compiling to JavaByteCode are supported: JVM and Java are equivalent
(2) Required for JVM
(3) If it is an expression, it must be resolvable at compilation time

RULES

[RULE <name>]
WHEN <precondition> [{AND | OR} […]]
THEN <actions> [; …]
[{ ELSE <actions> [; …] | IF (<future-condition>) [{AND | OR} […]] }]
[USE <alias>] [; …]

PreConditions
[[ANY | ALL] <group_name>] <expression>

Actions
{<script> | <rule> | {<device> | <group>} = <expression>}
[AFTER <time_unit>] [ONERROR <action>]

Future-conditions (optional)
([[ANY | ALL | NONE] <group_name>] <expression>)
[{AFTER | WITHIN} <time>]

Aliases (optional)
USE <name> AS <expression>

Examples:

```
WHEN alarm IS ON
    THEN all_lights SET OFF

WHEN ANY contact IS OPEN AND alarm IS ON
    THEN MyTelegram SET “Danger: intruders at home”
     IF (alarm IS ON) AFTER 20s

WHEN ANY door IS open OR ANY window IS open
    THEN heater SET off ONERROR MyTelegram SET “Can not set heater ON”
    IF (ANY door IS open OR ANY window IS open) WITHIN 3m
```

DEVICES (SENSORS & ACTUATORS)

DEVICE(1) <name>
[INIT <(2)prop> = {<literal> | <expresion(3)>} [; …]]
DRIVER <driver>
[CONFIG name = {<literal> | <expression(3)>} [; …]]

Example:

```
DEVICE LightKitchen
    INIT
        groups  SET “lights, ground_floor”
    (3) value   SET “an string”.len() ABOVE 0
    DRIVER driverForLights
        CONFIG
            IP SET ”192.168.1.110”
```

(1) SENSOR and ACTUATOR are synonyms (alias) of DEVICE
(2) In MSP is one of following: ‘groups’, ‘value’, ‘delta’, ‘downtime’
(3) If it is an expression, it must be resolvable at compilation time

How to learn *Une*

Read “Une_language.pdf” and try the examples in the “examples” folder (they are ordered by difficulty) using the IDE (named “Glue”).

RESERVED KEYWORDS

AFTER	ALL	ANY	AS	CALL	CONFIG	DEVICE
DRIVER	ELSE	FROM	IF	INCLUDE	INIT	LANGUAGE
LIBRARY	NONE	ONERROR	ONSTART	ONSTOP	PRE	REQUIRED
RULE	SCRIPT	THEN	USE	WHEN	WITHIN	